

תשובות- מבחן בית

1. בחרתי לנהל את Staten בצד השרת באמצעות SessionId יחודי לכל משתמש.

בSession נשמרים הנתונים:

N

CurrentIndex

CurrentValues

BrowseBaseIndex

LastBrowseStartIndex

הקומבימציות לא נשמרות בזיכרון.

Staten נשמר דרך ICombinationSessionStore והמימוש הנוכחי הוא InMemoryCombinationSessionStore עם ConcurrentDictionary כדי לתמוך בכמה משתמשים במקביל בצורה בטוחה.

בחרתי בזב כי Client לא אמור להיות מקור האמת ולא אמור לחשב או לשמור לוגיקה עסקית.

2. ההבדל בין NextPermutation לPermutation Based Ind:

NextPermutation מקבל קומבינציה קיימת ומחשב ממנה את הקומבינציה הבאה בסדר לקסיקוגרפי. הוא מתאים לכפתור NEXT כי יש לנו כבר את הקומבינציה הנוכחית ורוצים להתקדם צעד אחד קדימה.

Permutation Based Index מחשב את הקומבינציה ישירות לפי מספר סידורי, בלי לחשב את כל הקומבינציות שלפניה. הוא מתאים לPagination, קפיצה לעמוד Last Page ו-Resize כי שם צריך להגיע ישירות לאינדקס מסוים.

Next → LexicographicNextPermutation

Browse/Page/Jump/Resize → PermutationByIndex

3. אם נכנסים כמה משתמשים בו זמנית כל משתמש מקבל SessionId משלו.

הSession נשמר ב <Guid, CombinationSession> ConcurrentDictionary ולכן כל משתמש עובד מול State נפרד:

User A → Session A

User B → Session B

User C → Session C

הם לא דורסים אחד את השני, גם אם הם עובדים במקביל.

Storen רשום כSingleton כי הוא צריך לשמור את הSession בין קריאות, HTTP אבל הוא thread-safe בזכות ConcurrentDictionary.

4. אם נרצה להוסיף Redis נוכל ליצור מחלקה חדשה RedisCombinationSessionStore שתממש

את ICombinationSessionStore.

ונחליף את הרישום בProgram.cs ל

```
builder.Services.AddSingleton<ICombinationSessionStore,
RedisCombinationSessionStore>();
```

5. NextPermutation טוב כשרוצים להתקדם צעד קדימה. אם רוצים להגיע לקומבינציה רחוקה צריך להריץ את NextPermutation n פעמים.
עבור $N=20$ מספר האפשרויות הוא $20!$ שזה 2,432,902,008,176,640,000
ולכן אם נרצה לחשב את הקומבינציה הנ"ל נצטרך להריץ את NextPermutation 2,432,902,008,176,640,000 פעמים!
לכן עבור Pagination וקפיצה לעמודים השתמשתי ב-PermutationByIndex שמחשב את הקומבינציה לפי האינדקס.